



RELATÓRIO TÉCNICO

GeoLog — Monitoramento de Frota com Persistência Poliglota

Empresa fictícia	LogiTech Express
Aplicação	GeoLog / Streamlit
Banco relacional	PostgreSQL
Banco geoespacial	MongoDB
Visualização	Folium + Plotly

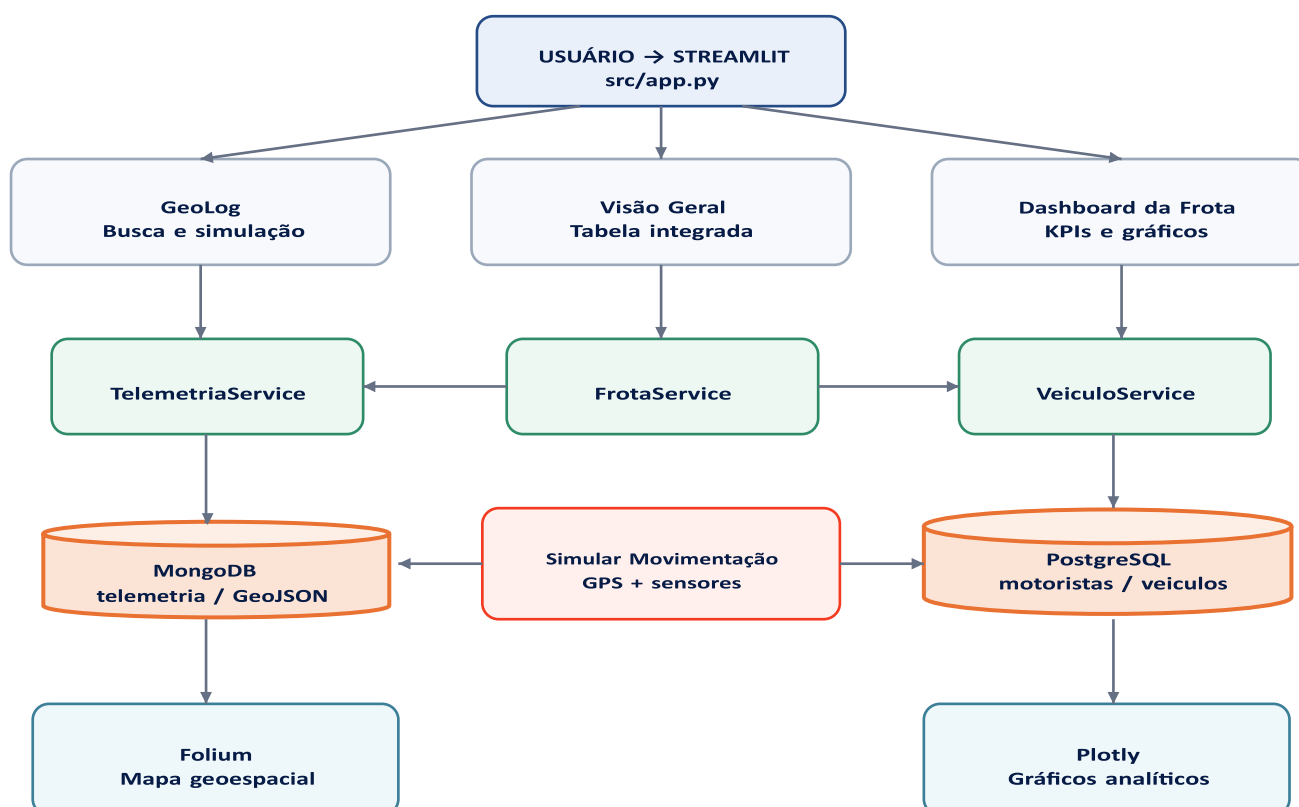
1. Visão geral

O GeoLog é uma aplicação Streamlit para monitoramento de uma frota logística. O projeto utiliza persistência poliglota: o PostgreSQL mantém os dados cadastrais e relacionais, enquanto o MongoDB armazena a telemetria geoespacial dos veículos.

A aplicação combina os dados dos dois bancos e apresenta buscas geográficas, uma visão integrada da frota, indicadores operacionais, gráficos analíticos e um mapa interativo.

2. Arquitetura poliglota

A arquitetura distribui cada tipo de dado para o mecanismo mais adequado e realiza a integração na camada de serviços da aplicação.



Responsabilidades dos componentes

Componente	Responsabilidade
Streamlit	Interface, navegação entre páginas, formulários, botões, KPIs e atualização da sessão.
TelemetriaService	Consultas ao MongoDB, agregação da última leitura por veículo, buscas geoespaciais e simulação de novas leituras.
VeiculoService	Consultas SQL para veículos, motoristas, placas, modelos e status.

Componente	Responsabilidade
FrotaService	Integração dos dados relacionais e de telemetria em memória usando pandas.
PostgreSQL	Dados estruturados de motoristas e veículos, com chave estrangeira entre motorista e veículo.
MongoDB	Coleção telemetria, contendo <code>veiculo_id</code> , localização GeoJSON, temperatura, velocidade e timestamp.
Folium	Mapa com ponto de referência, raio de busca e marcadores dos veículos.
Plotly	Gráficos de velocidade atual, status dos motoristas e histórico de temperatura.

3. Fluxo de dados

1. O usuário acessa uma página em `src/app.py`.
2. A página solicita os dados aos serviços da aplicação.
3. O `VeiculoService` consulta o cadastro no PostgreSQL.
4. O `TelemetriaService` consulta a última leitura de cada veículo no MongoDB.
5. O `FrotaService` cruza os resultados pelo `veiculo_id`.
6. A interface exibe os dados em tabelas, KPIs, gráficos e mapa.

A posição atual é obtida por uma agregação que ordena as leituras por veículo e timestamp decrescente e seleciona a mais recente. Isso evita exibir posições históricas como se fossem posições atuais.

4. Consultas geoespaciais

A página inicial oferece dois modos de consulta:

- Veículos dentro do raio: utiliza `$geoWithin` com `$centerSphere` e retorna os veículos contidos na área.
- Veículos mais próximos: utiliza `$geoNear`, calcula `distancia_metros` e retorna os veículos ordenados pela proximidade.

A coleção possui índice `2dsphere` em `location` e índice composto por `veiculo_id` e `timestamp` para apoiar as consultas.

5. Simulação de movimentação

O botão Simular Movimentação executa o seguinte fluxo:

1. Obtém a última leitura de cada veículo.
2. Aplica uma pequena variação aleatória à longitude e latitude.
3. Ajusta levemente velocidade e temperatura.
4. Insere uma nova leitura no MongoDB, preservando o histórico.

5. Limpa o cache de dados do Streamlit.
6. Refaz a consulta geográfica e atualiza o mapa na mesma sessão.

A simulação é controlada pela interface e não exige reinicialização do processo Streamlit.

6. Execução e infraestrutura

Os bancos são executados pelo Docker Compose:

- PostgreSQL: porta externa 5433.
- MongoDB: porta externa 27018.
- Volumes Docker: `postgres_data` e `mongo_data`.

Para iniciar a infraestrutura e a aplicação:

```
docker compose up -d  
  
.\.env\Scripts\python.exe src\db\seed.py  
  
.\.env\Scripts\python.exe -m streamlit run src\app.py
```

O seed recria o cadastro relacional e a coleção de telemetria com dados determinísticos para demonstração.

7. Considerações técnicas

- O cache de dados das páginas analíticas possui TTL de 30 segundos e pode ser invalidado pelo botão de atualização.
- As credenciais e URLs dos bancos são mantidas no `.env` e no `secrets.toml`, fora do controle de versão.
- O join poliglota é realizado em memória; cada banco continua responsável pelo seu próprio domínio de dados.
- A simulação é adequada para demonstração e testes de interface. Em produção, a geração de telemetria deveria ser substituída por um produtor real, como um dispositivo IoT, broker MQTT ou serviço de ingestão assíncrono.